TaskMate

Student Task Management System — Complete Technical Project Report

Academic Subject:	Computer Science & Engineering Mini Project
Tech Stack:	Python 3, Flask 3.1, SQLite 3, Jinja2, HTML5/CSS3
DevOps Tools:	Docker, Jenkins CI, pytest 9.1, Git & GitHub, Jira
Repository:	https://github.com/legendarydeep/STM
Live Local Endpoint:	http://127.0.0.1:5000
Target Environment:	Windows 11 / Linux / Containerized Environments

1. Executive Summary & Problem Statement

Modern undergraduate computer science students are tasked with managing concurrent lab sheets, continuous assessment tests, coding projects, and seminar presentations across multiple courses. Relying on paper notebooks or disparate chat groups leads to missed assignment cutoffs, poor academic prioritization, and cognitive overload.

TaskMate solves this challenge by providing a centralized, responsive web platform tailored for student workflows. It delivers real-time workload analytics (Total, Completed, Pending, and Overdue tasks), subject categorizations, fast modal creation, deadline calendar views, switchable table/card presentations, and light/dark theme switching with zero external database dependencies.

2. System Architecture & Tech Stack

The project follows the Model-View-Controller (MVC) architectural pattern implemented with a clean Flask Application Factory pattern (create_app):

Layer	Technology	Functionality & Architectural Role
Backend	Python 3 & Flask 3.1	Application factory, route dispatching, server-side data validation, and session management.
Database	SQLite 3 (Embedded)	Single-file relational engine (instance/tasks.db). Thread-safe connections, foreign key support, ACID transactions.
Frontend	HTML5, CSS3, Jinja2	Light/Dark design system with CSS custom properties, SVG circular donut progress chart, and modal dialogs.
Testing	pytest 9.1	10 automated unit and integration tests executing against isolated temporary SQLite databases.

DevOps	Docker & Jenkinsfile	python:3.11-slim container with unprivileged user; declarative CI pipeline generating JUnit XML reports.

3. Database Design & SQL Injection Prevention

The application uses an embedded SQLite database with the following table schema definition:

Column	Type	Constraints	Description
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique integer identifying each task record.
`title`	TEXT	NOT NULL, Max 150 chars	Title / headline of the academic task.
`subject`	TEXT	DEFAULT 'General'	Course name (e.g. Mathematics, Operating Systems).
`description`	TEXT	Nullable	Detailed task notes, links, or instructions.
`priority`	TEXT	CHECK: Low, Medium, High	Priority ranking for task scheduling.
`due_date`	TEXT	ISO Format: YYYY-MM-DD	Submission or exam target deadline.
`status`	TEXT	CHECK: Pending, Completed	Current lifecycle status of the task.
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auto-generated task creation timestamp.

Security Architecture: Preventing SQL Injection

All database operations enforce strict **parameterized queries** using question-mark (?) placeholders. Input strings are passed separately to the SQLite query engine as binary parameters, completely preventing malicious SQL execution via input fields:

```
# SECURE: Parameterized Query
db.execute("SELECT * FROM tasks WHERE subject = ? AND status = ?", (subj, stat))
```

4. Application Routing & Endpoints

Route / Endpoint	HTTP Method	Description & Business Logic
`/`	GET	Main Dashboard: computes metrics (Total, Completed, Pending, Overdue) and renders recent tasks and deadlines.
`/tasks`	GET	My Tasks directory: supports tabbed filters (All, Pending, Completed, Overdue), search query, and table/grid views.
`/calendar`	GET	Deadlines view: groups scheduled assignments chronologically by submission date.
`/tasks/new`	GET / POST	Task creation endpoint: executes server-side validation and creates a database row.
`/tasks/<id>/toggle`	POST	One-click fast toggle: flips task state between 'Completed' and 'Pending'.
`/tasks/<id>/edit`	GET / POST	Task modification: updates title, subject, priority, due date, description, and status.
`/tasks/<id>/delete`	POST	Deletion: removes task row from database with confirmation safeguard.
`/login & /logout`	GET / POST	Split-screen authentication portal with demo sign-in and session clearing.
`/settings`	GET / POST	Preferences page: manages student display name and theme mode.

5. Automated Testing & Verification Suite

Automated regression testing is implemented with **pytest** in tests/test_app.py. All 10 tests run against an isolated temporary database created via tempfile.mkstemp(), guaranteeing that automated test executions never corrupt or delete production data:

Test Function	Target Behavior Tested	Result
`test_home_page_loads`	Checks HTTP 200 and presence of all 4 metric cards.	PASSED
`test_create_task`	Verifies valid task creation and database insertion.	PASSED
`test_create_task_validation`	Asserts empty titles return HTTP 400 Bad Request.	PASSED
`test_update_task_status`	Tests status transition from Pending to Completed.	PASSED
`test_toggle_task`	Tests one-click completion toggle and reverse toggle.	PASSED
`test_update_task_details`	Verifies full editing of title, subject, priority, and date.	PASSED
`test_delete_task`	Verifies deletion and removal from SQLite database.	PASSED
`test_dashboard_counts_calculation`	Validates accurate calculation of Total, Pending, and Overdue.	PASSED

`test_task_filtering_and_tabs`	Validates that tab filters isolate Pending from Completed tasks.	PASSED
`test_calendar_and_login_views`	Verifies that Calendar and Login endpoints render HTTP 200.	PASSED

6. DevOps: Docker Containerization & Jenkins CI

A. Docker Containerization: The application is containerized using a slim base image (python:3.11-slim) with an unprivileged system user (appuser, UID 1000). A host bind mount (-v ${PWD}/instance:/app/instance) maps the local database directory to the container, ensuring that SQLite data persists across container restarts and updates.

B. Jenkins Continuous Integration: The declarative Jenkinsfile defines a three-stage automated pipeline: (1) Checkout source code from GitHub, (2) Install Python dependencies from requirements.txt, (3) Execute pytest --junitxml=test-results.xml. The Jenkins JUnit plugin archives the test report to track build health.

C. Jira Agile Planning: Project backlog and sprints are organized under key **STMS**, covering 8 user stories with Gherkin acceptance criteria (Given/When/Then), story point estimations, and Smart Commits tracking.

7. Essential Viva Voce Questions & Answers

Q1: What is Flask and why is it called a micro-framework?

Flask is a lightweight Python web framework based on Werkzeug WSGI and Jinja2 templating. It is called 'micro' because it provides core routing and templating without enforcing specific database engines or form libraries, leaving architectural choices to the developer.

Q2: Why did you choose SQLite instead of MySQL or PostgreSQL for this project?

SQLite is serverless, zero-configuration, and stores the entire database in a single portable file (instance/tasks.db). It supports full ACID transactions and eliminated the need to maintain an external database server, making it lightweight and ideal for practical demonstrations.

Q3: How do you prevent SQL Injection in your database queries?

By strictly using parameterized queries with question-mark (?) placeholders. User inputs are treated as literal values rather than executable SQL statements, completely preventing injection attacks.

Q4: What is the Application Factory pattern (create_app)?

It is a design pattern where the Flask application instance is created inside a function rather than as a global variable. This enables creating isolated application instances with different configurations (e.g. temporary in-memory database for testing vs file-based database for production).

Q5: What is the difference between a Docker image and a Docker container?

A Docker image is a read-only, immutable template containing the OS, libraries, and code (similar to a Class in OOP). A Docker container is a running, stateful instance instantiated from that image (similar to an Object).

Q6: Why is a Docker volume essential for SQLite in containerized environments?

Containers are ephemeral; any files written inside a container are deleted when the container is stopped or removed. A Docker Volume or bind mount maps a host directory to /app/instance so database records persist permanently.

Q7: What does Continuous Integration (CI) mean in this project?

CI is the practice of automatically triggering builds and automated pytest suites whenever code is pushed to GitHub. In our Jenkinsfile, any failing test marks the build as failed, preventing regressions from reaching production.

Q8: Why did your automated tests use temporary databases instead of tasks.db?

If tests used the live database, running pytest would corrupt or wipe real user tasks. Using Python's tempfile.mkstemp() creates an isolated database for each test session and cleans it up during teardown.

8. Project Demonstration Reference

To run the project in VS Code during your examination: (1) Open VS Code terminal (`Ctrl + ~`), (2) Execute python app.py, (3) Open browser at http://127.0.0.1:5000. To run automated tests, execute python -m pytest in the terminal.